

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-213БВ-25

Студент: Лапшина М. Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 04.10.26

Москва, 2026

Постановка задачи

Цель работы

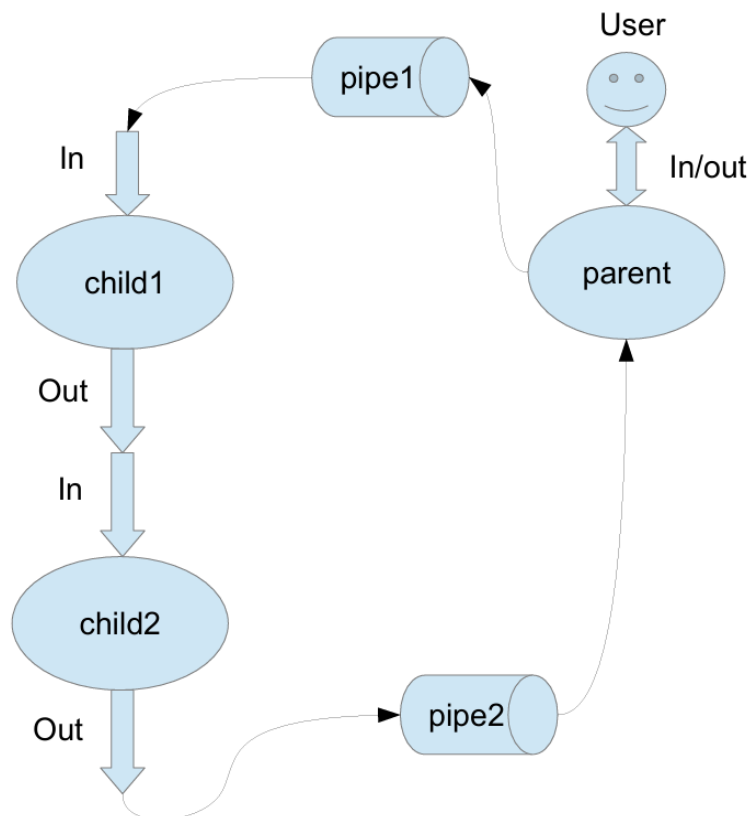
Приобретение практических навыков в:

- Управление процессами в ОС.
- Обеспечение обмена данных между процессами посредством каналов.

Вариант 14.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Группа вариантов



Родительский процесс создает два дочерних процесса. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Child1 и Child2 можно «соединить» между собой дополнительным каналом. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child1 и child2 производят работу над строками. Child2 пересылает результат своей работы родительскому процессу. Родительский процесс полученный результат выводит в стандартный поток вывода.

14 вариант. Child1 переводит строки в нижний регистр. Child2 убирает все задвоенные пробелы.

Общий метод и алгоритм решения

В программе создаются три неименованных канала `pipe`, которые используются для последовательной передачи данных между процессами. Сначала родительский процесс создаёт три канала и с помощью системного вызова `fork()` создаёт два дочерних процесса.

Первый дочерний процесс получает данные из первого канала, преобразует символы строки в нижний регистр и передаёт результат через третий канал. Второй дочерний процесс получает данные из третьего канала, удаляет повторяющиеся пробелы и передаёт обработанную строку через второй канал обратно родительскому процессу. Родительский процесс считывает строки с клавиатуры, передаёт их в первый канал, получает обработанные строки из второго канала и выводит их на экран.

Для перенаправления стандартного ввода и вывода дочерних процессов используется системный вызов `dup2()`. После завершения работы родительский процесс ожидает завершения дочерних процессов с помощью `waitpid()`.

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс.
- `int pipe(int *fd)`; – Создает неименованный канал для обмена данными между процессами.
- `int dup2(int oldfd, int newfd)`; – копирует файловый дескриптор `oldfd` в `newfd`.
- `int close(int fd)` – закрывает файловый дескриптор.
- `pid_t waitpid(pid_t pid, int *status, int options)` - Ожидание завершения дочернего процесса.
- `exec1(...)` – заменяет текущий программный код процесса другой программой.

В начале работы родительский процесс создаёт три канала: `pipe1`, `pipe2` и `pipe3`. Каждый канал имеет два файловых дескриптора: один используется для чтения, другой для записи. Первый канал предназначен для передачи данных от родительского процесса к первому дочернему процессу, третий канал используется для передачи данных от первого дочернего процесса ко второму, а второй канал предназначен для передачи окончательного результата от второго дочернего процесса обратно родительскому процессу.

После создания каналов родительский процесс дважды вызывает системный вызов `fork()`. В результате создаются два дочерних процесса. Для первого дочернего процесса после `fork()` выполняется программа `child1`, а для второго дочернего процесса выполняется программа `child2`. Для запуска этих программ используется системный вызов `exec1()`, который заменяет программный код текущего дочернего процесса на код соответствующей программы.

Для организации правильной передачи данных между процессами используется системный вызов `dup2()`. У первого дочернего процесса стандартный ввод перенаправляется на конец `pipe1`, предназначенный для чтения, а стандартный вывод перенаправляется на конец `pipe3`, предназначенный для записи. Поэтому `child1` может читать строки из стандартного ввода, фактически получая их от родительского процесса, и выводить обработанные строки в `pipe3`.

У второго дочернего процесса стандартный ввод перенаправляется на конец `pipe3`, предназначенный для чтения, а стандартный вывод перенаправляется на конец `pipe2`, предназначенный для записи. Таким образом, второй дочерний процесс получает результат работы `child1`, обрабатывает его и передаёт результат родительскому процессу.

Родительский процесс считывает строки с клавиатуры с помощью функции `getline()`. Данная функция позволяет получать строки произвольной длины. Полученная строка записывается в `pipe1` с помощью системного вызова `write()`.

Первый дочерний процесс считывает строку из стандартного ввода с помощью `getline()`. Для каждого символа строки вызывается функция `tolower()`, которая переводит символ в нижний регистр. После обработки строка передаётся в стандартный вывод, который предварительно был перенаправлен в `pipe3`. Таким образом, результат работы первого дочернего процесса поступает второму дочернему процессу.

Второй дочерний процесс также получает строку из стандартного ввода. Он последовательно просматривает все символы строки и удаляет повторяющиеся пробелы. В результате последовательность нескольких пробелов заменяется одним пробелом. После обработки строка передаётся в стандартный вывод, перенаправленный в `pipe2`.

Родительский процесс получает обработанную строку из `pipe2` и выводит её на экран. После завершения передачи данных родительский процесс закрывает использованные файловые дескрипторы и с помощью системного вызова `waitpid()` ожидает завершения обоих дочерних процессов. Это позволяет корректно завершить работу всех процессов программы.

При работе с системными вызовами в программе выполняется проверка их возвращаемых значений. Например, проверяется успешность создания каналов с помощью `pipe()`, создания дочерних процессов с помощью `fork()`, перенаправления файловых дескрипторов с помощью `dup2()`, запуска дочерних программ с помощью `exec1()` и записи данных в канал с помощью `write()`. При возникновении ошибки используется функция `perror()`, которая выводит сообщение, соответствующее значению `errno`. После обнаружения критической ошибки выполнение соответствующего процесса прекращается.

Код программы

Parent.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main(void) {
    int pipe1[2];
    int pipe2[2];
    int pipe3[2];

    if (pipe(pipe1) == -1) {
        perror("pipe1");
        return 1;
    }

    if (pipe(pipe2) == -1) {
        perror("pipe2");
        close(pipe1[0]);
        close(pipe1[1]);
        return 1;
    }
```

```

if (pipe(pipe3) == -1){
    perror("pipe3");
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    return 1;
}

pid_t pid1 = fork();

if (pid1 == -1){
    perror("fork child1");
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    close(pipe3[0]);
    close(pipe3[1]);
    return 1;
}

if (pid1 == 0){

    if (dup2(pipe1[0], STDIN_FILENO) == -1){
        perror("dup2 child1 stdin");
        _exit(1);
    }

    if (dup2(pipe3[1], STDOUT_FILENO) == -1){
        perror("dup2 child1 stdout");
        _exit(1);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    close(pipe3[0]);
    close(pipe3[1]);

    execl("./child1", "child1", (char *)NULL);

    perror("execl child1");
    _exit(1);
}

pid_t pid2 = fork();

if (pid2 == -1){
    perror("fork child2");
    close(pipe1[0]);

```

```

        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        close(pipe3[0]);
        close(pipe3[1]);
        return 1;
    }

    if (pid2 == 0){

        if (dup2(pipe3[0], STDIN_FILENO) == -1) {
            perror("dup2 child2 stdin");
            _exit(1);
        }

        if (dup2(pipe2[1], STDOUT_FILENO) == -1) {
            perror("dup2 child2 stdout");
            _exit(1);
        }

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        close(pipe3[0]);
        close(pipe3[1]);

        execl("./child2", "child2", (char *)NULL);

        perror("execl child2");
        _exit(1);
    }

    close(pipe1[0]);
    close(pipe3[0]);
    close(pipe3[1]);
    close(pipe2[1]);

    FILE *output = fdopen(pipe2[0], "r");
    if (output == NULL){
        perror("fdopen");

        close(pipe1[1]);
        close(pipe2[0]);

        waitpid(pid1, NULL, 0);
        waitpid(pid2, NULL, 0);
        return 1;
    }

    char *line = NULL;
    size_t size = 0;

```

```

while (getline(&line, &size, stdin) != -1){

    size_t length = 0;

    while(line[length] != '\0')
        length++;

    size_t written_total = 0;

    while (written_total < length){

        ssize_t written = write(pipel[1], line + written_total, length -
written_total);

        if (written == -1){
            perror("write pipel");
            free(line);
            fclose(output);

            waitpid(pid1, NULL, 0);
            waitpid(pid2, NULL, 0);
            return 1;
        }

        written_total += (size_t)written;
    }

    char *result = NULL;
    size_t result_size = 0;

    if (getline(&result, &result_size, output) == -1){
        free(result);
        break;
    }

    printf("%s", result);

    free(result);
}

free(line);

close(pipel[1]);
close(output);

if (waitpid(pid1, NULL, 0) == -1){
    perror("waitpid child1");
    return 1;
}

if (waitpid(pid2, NULL, 0) == -1){
    perror("waitpid child2");
    return 1;
}

```

```
}  
    return 0;  
}
```

Child1.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <ctype.h>  
  
int main(void){  
    char *line = NULL;  
    size_t size = 0;  
  
    setvbuf(stdout, NULL, _IONBF, 0);  
  
    while (getline(&line, &size, stdin) != -1) {  
        for (size_t i = 0; line[i] != '\0'; ++i)  
            line[i] = (char)tolower((unsigned char)line[i]);  
  
        fputs(line, stdout);  
    }  
  
    free(line);  
    return 0;  
}
```

Child2.c

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main(void){  
    char *line = NULL;  
    size_t size = 0;  
  
    setvbuf(stdout, NULL, _IONBF, 0);  
  
    while (getline(&line, &size, stdin) != -1) {  
        size_t read_index = 0;  
        size_t write_index = 0;  
        int previous_space = 0;  
  
        while (line[read_index] != '\0') {  
            if (line[read_index] == ' ') {  
                if (!previous_space) {  
                    line[write_index++] = line[read_index];  
                    previous_space = 1;  
                }  
            } else {  
                line[write_index++] = line[read_index];  
                previous_space = 0;  
            }  
  
            ++read_index;  
        }  
    }  
}
```

```

    }

    line[write_index] = '\0';
    fputs(line, stdout);
}

free(line);
return 0;
}

```

Протокол работы программы

Тестирование:

```

marikl\DESKTOP-4IVBKII:/mnt/c/Users/User/Documents/GitHub/lab_OS/lab_1$ gcc parent.c -o parent
marikl\DESKTOP-4IVBKII:/mnt/c/Users/User/Documents/GitHub/lab_OS/lab_1$ gcc child1.c -o child1
marikl\DESKTOP-4IVBKII:/mnt/c/Users/User/Documents/GitHub/lab_OS/lab_1$ gcc child2.c -o child2
marikl\DESKTOP-4IVBKII:/mnt/c/Users/User/Documents/GitHub/lab_OS/lab_1$ ./parent
HELLO  WORLD  !
hello world !
THIS  is      A      TeST...
this is a test...
I LoVe C    ?
i love c ?
marikl\DESKTOP-4IVBKII:/mnt/c/Users/User/Documents/GitHub/lab_OS/lab_1$ |

```

```

marikl\DESKTOP-4IVBKII:/mnt/c/Users/User/Documents/GitHub/lab_OS/lab_1$ ./child1
HMMMMMM...
hmmmmmm...
I am  MARIA
i am  maria
marikl\DESKTOP-4IVBKII:/mnt/c/Users/User/Documents/GitHub/lab_OS/lab_1$ ./child2
A O BBBB   NNN
A O BBBB   NNN
      HELLo,      WORLD !
HELLo, WORLD !
marikl\DESKTOP-4IVBKII:/mnt/c/Users/User/Documents/GitHub/lab_OS/lab_1$ |

```

Вывод

В ходе лабораторной работы была реализована программа взаимодействия родительского и двух дочерних процессов с использованием неименованных каналов pipe. Родительский процесс передаёт строки первому дочернему процессу, после чего данные последовательно обрабатываются двумя дочерними процессами и возвращаются родителю. В работе были изучены системные вызовы fork, pipe, dup2, close, execl и waitpid. Основная сложность заключалась в правильной настройке направлений передачи данных.